

Trabalho 6 - Árvores Binárias de Pesquisa

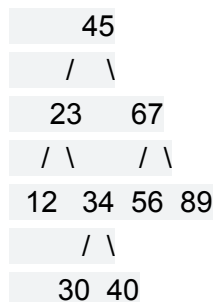
Identificação do Aluno

- **Nome:** Rafael Lima Sant'Ana
- **Matrícula:** 251035659
- **Disciplina:** Estruturas de Dados e Algoritmos 1 (EDA 1)
- **Turma:** T04
- **Professor:** MSc. Filipe Emídio Tôrres
- **Instituição:** Universidade de Brasília (UnB) - Faculdade do Gama

Questão 1: Construção e Propriedades

Dada a sequência de inserção de chaves inteiras em uma ABP inicialmente vazia: 45, 23, 67, 12, 34, 56, 89, 30, 40.

1. Desenhe a árvore resultante após todas as inserções.



2. Identifique a raiz, os nós folhas e os nós internos.

- **Raiz:** 45.
- **Folhas:** 12, 30, 40, 56, 89.
- **Nós internos:** 45, 23, 67, 34.

3. Determine a altura da árvore (considerando raiz no nível 0).

A altura da árvore é 3 (Nível 0: 45 | Nível 1: 23, 67 | Nível 2: 12, 34, 56, 89 | Nível 3: 30, 40).

Questão 2: Análise de Percursos

Utilizando a árvore gerada na Questão 1, apresente os resultados dos seguintes

percursos:

4. Percurso em Pré-Ordem (Raiz, Esquerda, Direita).

45, 23, 12, 34, 30, 40, 67, 56, 89.

5. Percurso em Em-Ordem (Esquerda, Raiz, Direita).

12, 23, 30, 34, 40, 45, 56, 67, 89.

6. Percurso em Pós-Ordem (Esquerda, Direita, Raiz).

12, 30, 40, 34, 23, 56, 89, 67, 45.

7. Explique por que o percurso Em-Ordem é particularmente útil em Árvores Binárias de Pesquisa.

O percurso Em-Ordem é útil em Árvores Binárias de Pesquisa porque visita e retorna os nós em **ordem crescente** de suas chaves, permitindo a fácil extração de dados ordenados.

Questão 3: Implementação em C

8. A definição da estrutura TNoBin.

```
typedef struct TNoBin {  
    int chave;  
    struct TNoBin *esq;  
    struct TNoBin *dir;  
} TNoBin;
```

9. Uma função para criar um novo nó.

```
TNoBin* criarNo(int chave) {  
    TNoBin* novoNo = (TNoBin*)malloc(sizeof(TNoBin));  
    if (novoNo != NULL) {  
        novoNo->chave = chave;  
        novoNo->esq = NULL;  
        novoNo->dir = NULL;  
    }  
    return novoNo;  
}
```

10. Uma função recursiva para inserção de elementos.

```
TNoBin* inserir(TNoBin* raiz, int chave) {  
    if (raiz == NULL) {
```

```

    return criarNo(chave);
}
if (chave < raiz->chave) {
    raiz->esq = inserir(raiz->esq, chave);
} else if (chave > raiz->chave) {
    raiz->dir = inserir(raiz->dir, chave);
}
return raiz;
}

```

11. Uma função de busca que retorne o endereço do nó ou NULL.

```

TNoBin* buscar(TNoBin* raiz, int chave) {
    if (raiz == NULL || raiz->chave == chave) {
        return raiz;
    }
    if (chave < raiz->chave) {
        return buscar(raiz->esq, chave);
    }
    return buscar(raiz->dir, chave);
}

```

Questão 4: Caso Prático - Sistema de Inventário de Laboratório

Modele a estrutura do nó para este sistema e escreva uma função em C que receba a raiz da árvore e um código de patrimônio, retornando o nome do equipamento caso ele exista, ou uma mensagem de "Não encontrado".

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct NoEquipamento {
    int codPatrimonio;
    char nome[100];
}

```

```

    struct NoEquipamento *esq;
    struct NoEquipamento *dir;
} NoEquipamento;

void buscarEquipamento(NoEquipamento* raiz, int codigo) {
    if (raiz == NULL) {
        printf("Não encontrado\n");
        return;
    }
    if (codigo == raiz->codPatrimonio) {
        printf("Equipamento: %s\n", raiz->nome);
        return;
    }
    if (codigo < raiz->codPatrimonio) {
        buscarEquipamento(raiz->esq, codigo);
    } else {
        buscarEquipamento(raiz->dir, codigo);
    }
}

```

Questão 5: Análise de Balanceamento

Considere duas sequências de inserção para os mesmos dados:

- Sequência A: 10, 20, 30, 40, 50
- Sequência B: 30, 10, 40, 20, 50

Explique, tecnicamente, qual a diferença estrutural entre as árvores resultantes e como isso afeta a complexidade de tempo da operação de busca em cada uma delas.

A **Sequência A** insere os dados de forma já ordenada. Isso faz com que a árvore não se ramifique e se torne uma árvore "degenerada", comportando-se estruturalmente como uma lista encadeada, onde todos os nós ficam apenas à direita. Como resultado, a complexidade de tempo da operação de busca no pior caso torna-se **O(n)** (linear).

A **Sequência B** insere os valores de forma mais intercalada. Isso resulta em uma árvore estruturalmente balanceada (ou próxima disso), permitindo ramificações adequadas para a esquerda e para a direita. O tempo de busca nesta árvore se aproxima do ideal logarítmico, reduzindo a complexidade de tempo para **O(log n)**, o que otimiza expressivamente a performance em sistemas maiores.